

BridgeAid

Community Resource & Opportunity Navigator

Python Final Project Documentation

Project 2 — GitHub Portfolio + CTI Student Spotlight

Core idea: Help a person move from “What can I realistically do?” to a ranked set of relevant educational, training, employment, and community-support options.

Important scope: Version 1 is a terminal-based decision-support application. It does not claim to make legal, medical, immigration, or financial decisions. It only matches user-provided information against a structured set of sample resources and explains the matching rules.

Prepared as a coding blueprint — start coding only after understanding this document.

1. Project Goal

Build a terminal-based Python application that helps people discover resources and opportunities that fit their current situation. The user creates a profile, enters constraints and goals, the program checks available resources, calculates a transparent match score, ranks the results, and shows missing requirements plus suggested next actions.

The project is deliberately designed to reinforce the concepts from the Python Final Project Guide: classes and objects, lists, dictionaries, encapsulation, inheritance, method overriding, input validation, separation of menu/application logic, and a complete working workflow.

2. Why This Is a Strong Second Python Project

- It solves a recognizable access problem: people often have opportunities around them but do not know which ones fit their constraints.
- It is not another generic CRUD manager. The core feature is a matching engine with explainable rules.
- It can be completed as a terminal application without needing a website, API, database, or paid AI service.
- It creates a natural path to later development: persistent storage, a web interface, real datasets, APIs, multilingual support, and optional AI-assisted explanations.
- It gives a strong CTI demo: one user profile goes in; several ranked, explained recommendations come out.

3. Version 1 Scope

Build these first:

- Create, view, search, update, and delete user profiles.
- Add, view, search, update, and delete resources/opportunities.
- Assign resource types and requirements.
- Match one user profile against available resources.
- Calculate a transparent match score.
- Show why a resource matched or did not match.
- Show missing requirements and practical next actions.
- Show a ranked recommendation report.
- Validate menu choices, IDs, numeric values, categories, and required fields.
- Use in-memory lists and dictionaries; no file saving is required for Version 1.

4. Main Menu

The terminal menu should be simple and understandable:

```
===== BridgeAid =====
1. Add User Profile
2. View Profiles
3. Search Profile
4. Update Profile
5. Delete Profile
6. Add Resource
7. View Resources
```

- 8. Search Resource
- 9. Match a Profile
- 10. View Recommendation Report
- 11. Dashboard
- 0. Exit

5. Data Model

Object	Purpose	Important attributes
UserProfile	Stores one person's situation and goals.	profile_id, name, age, education, skills, languages, budget, goals, constraints
Resource	Parent class for any opportunity/resource.	resource_id, title, provider, requirements
EducationResource	Training/course/education opportunity.	inherits Resource; education-specific requirements
EmploymentResource	Job/internship/apprenticeship opportunity.	inherits Resource; employment-specific requirements
SupportResource	Community/service/support resource.	inherits Resource; support-specific requirements
BridgeAid	Connects profiles and resources; owns the matching workflow.	profiles, resources, match results

6. Class Relationships

The relationships are intentionally similar in learning value to the course project, but the domain is completely different.

BridgeAid

- |-- stores many UserProfile objects
- |-- stores many Resource objects
- |-- runs the matching process

Resource <parent class>

- |-- EducationResource <child>
- |-- EmploymentResource <child>
- |-- SupportResource <child>

UserProfile

- |-- stores goals and constraints
- |-- is compared with Resource requirements

7. Class Guide

UserProfile Class

Represents one person using the system.

Attribute	Description
profile_id	Unique ID such as U001.
name	User's name.
age	Age used only when an opportunity has an age requirement.
education	Highest completed education level.

Attribute	Description
skills	List of skills, such as Python or English.
languages	Dictionary of language -> level, such as English -> B1.
budget	Available budget for the opportunity, if relevant.
goals	List of goals such as IT, training, or employment.
constraints	List of limitations such as low budget or schedule restriction.

Methods

- `get_id()` — returns the profile ID.
- `get_name()` — returns the person's name.
- `set_budget(value)` — updates the budget after validation.
- `add_skill(skill)` — adds a skill without creating duplicates.
- `add_goal(goal)` — adds a goal.
- `add_constraint(constraint)` — records a relevant constraint.
- `display_info()` — shows a readable profile summary.

Resource Class

Parent class for opportunities and community resources.

Attribute	Description
resource_id	Unique ID such as R001.
title	Name of the opportunity/resource.
provider	Organization or provider name.
category	General category.
requirements	Dictionary of requirements.
benefits	List describing what the resource offers.

Methods

- `is_eligible(profile)` — checks the general requirements.
- `calculate_match_score(profile)` — calculates a basic match score.
- `get_missing_requirements(profile)` — returns unmet requirements.
- `display_info()` — displays resource information.
- `recommendation_message(score)` — gives a general explanation.

EducationResource Class

Child class of Resource for courses, training programs, and educational opportunities.

- Overrides `display_info()` to identify the resource as education/training.
- Overrides `recommendation_message(score)` to provide education-specific feedback.
- May check education level or language requirements.

EmploymentResource Class

Child class of Resource for jobs, internships, apprenticeships, or entry pathways.

- Overrides `display_info()` to identify the resource as employment-related.
- Overrides `recommendation_message(score)` with employment-specific feedback.
- May check skill requirements and minimum language level.

SupportResource Class

Child class of Resource for community support services.

- Overrides `display_info()` to show the type of support.
- Overrides `recommendation_message(score)` to emphasize access and missing conditions.
- May use different matching rules because support resources may not require skills or education.

BridgeAid Class

Controls the whole application and connects profiles, resources, and matching.

- `add_profile(profile)` — stores a `UserProfile` in a dictionary using `profile_id`.
- `search_profile(keyword)` — searches by ID or name.
- `update_profile(profile_id, ...)` — updates selected profile information.
- `delete_profile(profile_id)` — removes a profile.
- `add_resource(resource)` — stores a `Resource` object.
- `search_resource(keyword)` — searches resources by ID, title, category, or provider.
- `match_profile(profile_id)` — evaluates all resources for the selected profile.
- `rank_matches(matches)` — sorts matching resources from strongest to weakest.
- `show_report(profile_id)` — displays a complete recommendation report.
- `get_dashboard()` — shows totals and basic system statistics.

8. Matching Logic — The Technical Heart of the Project

The most important part of BridgeAid is not storing data. It is the matching process. Version 1 should use transparent rules rather than a black-box AI model.

Suggested scoring model: start from 100 points and subtract points for unmet important requirements, while adding points for strong matches. Keep the exact weights simple enough to explain in the demo.

Example categories:

- Goal/category match
- Skill match
- Language match
- Education match
- Budget compatibility
- Constraint compatibility

The program must also return reasons, not only a number. For example: “Python skill matches”, “English level meets requirement”, “German requirement is B1 but profile is A2”, and “budget is within the stated limit.”

9. Match Result

You may use a small `MatchResult` class if you want cleaner code. It is optional for the core OOP requirement.

Attribute	Description
profile_id	The user being evaluated.
resource_id	The evaluated resource.
score	Numeric match score.
matched_requirements	Requirements the profile satisfies.
missing_requirements	Requirements the profile does not satisfy.
next_actions	Suggested practical next steps.

10. Complete Workflow

The main workflow should demonstrate the entire project from input to recommendation.

Step	Action	Example
Step 1	Create a profile	U001, education, skills, languages, budget, goals, constraints.
Step 2	Add resources	Add several education, employment, and support resources.
Step 3	Run matching	Select U001 and evaluate all available resources.
Step 4	Calculate scores	Compare the profile with each resource's requirements.
Step 5	Explain results	Show matched and missing requirements.
Step 6	Rank results	Show the strongest options first.
Step 7	Generate actions	Show what the user could do next to become eligible.

11. Example Scenario

User: U001 — 20 years old, high-school graduate, Python beginner, English B1, German A2, low budget, goal = IT career.

Resource A: Free beginner Python training — no language certificate required.
Possible result: 92% match.

Resource B: IT training program — requires German B1.
Possible result: 70% match.
Missing requirement: German B1.
Next action: improve German from A2 to B1.

Resource C: Paid advanced course — budget requirement exceeds profile budget.
Possible result: low match.
Reason: budget mismatch.

The exact scores above are examples only. During implementation, you must define and document your own scoring rules.

12. Input Validation and Error Handling

- Check that menu choices are valid.

- Check that profile IDs and resource IDs are unique when creating objects.
- Check that referenced profile/resource IDs exist before using them.
- Check that age and budget are numeric and within sensible ranges.
- Check that language levels use an accepted format such as A1, A2, B1, B2, C1, C2.
- Check that required fields are not empty.
- Check that duplicate skills, goals, and constraints are not added unnecessarily.
- Show clear error messages instead of allowing the application to crash.
- Handle empty search results clearly.

13. Python Concepts This Project Must Practice

Course concept	Where BridgeAid uses it
Lists	skills, goals, constraints, benefits, matched requirements, next actions.
Dictionaries	profiles by profile_id, resources by resource_id, language levels, requirement data.
Classes / objects	UserProfile, Resource, child Resource classes, BridgeAid, optional MatchResult.
Encapsulation	Use at least one private attribute with getter/setter or controlled update method.
Inheritance	EducationResource, EmploymentResource, and SupportResource inherit from Resource.
Method overriding	Child resource classes override display_info() and/or recommendation_message().
Validation	Menu choices, IDs, numeric values, categories, language levels, required fields.
Separation of concerns	Keep menu/input flow separate from BridgeAid business logic as much as practical.
Sorting/filtering	Rank resources by match score and filter/search resources.
Functions	Use helper functions for repeated validation or formatting.
Exception handling	Handle expected input conversion errors such as invalid integer input.

14. Suggested Project Structure

```

bridgeaid/
├── main.py
├── models.py
├── bridgeaid.py
├── matching.py
├── validation.py
├── sample_data.py
├── tests.py
└── README.md

```

Do not create this entire structure on day one if it makes the project confusing. You can begin with a few files and split modules when the code becomes large enough to justify it.

15. Two Required Creative Features

Choose these two as the official custom features for the project:

Feature	Description
Explainable Match Report	Every recommendation shows the score, matched requirements, missing requirements, and the reasons behind the result.

Feature	Description
Next-Step Bridge Plan	For a low or medium match, the program turns missing requirements into a short list of next actions.

Optional later features can include a comparison view, category statistics, multilingual labels, or a simple progress simulation. Do not add them until the main project is complete.

16. README.md Requirements

- Project title: BridgeAid — Community Resource & Opportunity Navigator.
- Your full name.
- Problem statement and target user.
- What the program does.
- How to run the project.
- Example workflow.
- Class list and responsibilities.
- Where encapsulation is used.
- Where inheritance and method overriding are used.
- Explanation of the matching algorithm.
- Explanation of the two creative features.
- Known limitations and what could be improved in a future version.

17. CTI Student Spotlight Demo — Suggested 5-7 Minute Flow

Time	What to show
0:00-0:45	Problem: people can struggle to identify which opportunities fit their actual constraints.
0:45-1:15	Introduce BridgeAid and explain the idea in one sentence.
1:15-2:15	Show the main menu and create a user profile.
2:15-3:15	Add or load several resources from the sample data.
3:15-4:30	Run matching and show ranked results.
4:30-5:15	Open one result and explain matched/missing requirements.
5:15-6:15	Show the code: classes, inheritance, overriding, lists/dictionaries, validation.
6:15-7:00	Explain the two custom features and future expansion.

18. Suggested Self-Assessment Rubric

Category	Target
OOP design	Classes, relationships, encapsulation, inheritance, overriding — 30
Required features	Profiles, resources, matching, ranking, report — 20

Category	Target
Lists and dictionaries	Used naturally — 10
Validation and error handling	Common wrong inputs handled — 10
Two creative features	Explainable report + next-step bridge plan — 10
Code readability and organization	Clear names, responsibilities, structure — 5
README	Complete and understandable — 5
Video explanation	5+ minutes and own code explained — 10
Total	100

19. Final Checklist Before You Call It Finished

- ☐ The program runs from main.py.
- ☐ I created at least four meaningful classes.
- ☐ I used lists and dictionaries where they make sense.
- ☐ I used encapsulation in at least one class.
- ☐ I used inheritance and method overriding.
- ☐ I can create, view, search, update, and delete profiles.
- ☐ I can create and manage resources.
- ☐ I can run a profile-resource matching process.
- ☐ The program calculates a transparent match score.
- ☐ The report explains why a resource matched.
- ☐ The program identifies missing requirements.
- ☐ The program generates next actions.
- ☐ The program handles common wrong inputs.
- ☐ I added the two custom features.
- ☐ My README is complete.
- ☐ My 5+ minute demo explains my own code.

20. Coding Order — Do Not Build Everything at Once

1. Create UserProfile and make profile creation/display work.
2. Create Resource and one child class. Test inheritance and overriding.
3. Add the other child resource types.
4. Create BridgeAid and dictionaries for profiles/resources.
5. Implement add/search/update/delete operations.
6. Implement requirement checking.
7. Implement match scoring.
8. Implement missing-requirement and next-action generation.
9. Implement ranking and recommendation report.

10. Add validation and error handling.
11. Add sample data for fast testing.
12. Write tests for important matching rules.
13. Clean the code and prepare README.
14. Record the demo only after the complete workflow works.

21. Scope Rules — What NOT to Build Yet

- Do not build a website in Version 1.
- Do not add a database before the terminal version works.
- Do not connect a paid AI API just to make the project look advanced.
- Do not scrape real websites in the first version.
- Do not claim the recommendations are legally, financially, medically, or immigration-professionally authoritative.
- Do not add ten extra features before the main matching workflow works.
- Do not copy another student's implementation. The design and code must remain your own.

22. Future Development Path

After Version 1 is complete, the project can grow without throwing away its core logic:

- Version 1: Python CLI + in-memory data + rule-based matching
- Version 2: persistent data storage
- Version 3: GUI or web interface
- Version 4: real verified datasets / APIs
- Version 5: multilingual support
- Version 6: optional AI-assisted natural-language explanations

23. Final Reminder

The goal is not to make the biggest project. The goal is to make a complete, understandable project that clearly demonstrates your Python skills. You should be able to explain every class, attribute, method, matching rule, list, dictionary, inheritance relationship, and validation decision in your own words.

Build the core first. Make it work. Then make it impressive.